

Capstone Project – Student Guide

CS5998 | Master of Data Science & Artificial Intelligence

6 Credits

1. What this capstone is (and is not)

The capstone project is an **individual, applied DS & AI project** carried out during **Semester 3**.

It is:

- A chance to practise **end-to-end problem solving**
- Focused on **realistic data, methods, and systems**
- Evaluated primarily through **what you build and justify**

It is **not**:

- A supervised MSc thesis
- A literature survey
- A small coding exercise
- A group project
- A research project

You are responsible for **scoping, execution, and time management**.

2. What makes a project acceptable

Your project **must** demonstrate all of the following:

- A **clear, real-world DS/AI problem**
- Use of **real or realistic data**
- An **end-to-end pipeline or system**, or a **comparative analysis using experimental evaluation (not just a literature survey)** with sufficient scope and depth.
- **Evaluation with justification**

3. How project scope is defined

To avoid vague or over/under-scoped projects, you will define your project using **bounded choices**.

You must explicitly specify:

- **Problem type** (e.g. prediction, classification, detection, recommendation, etc)
- **Data type** (e.g. tabular, text, image, graph, time series, multimodal)
- **Technique category** (e.g. ML, DL, data mining, analytics, computer vision, NLP, etc)
- **System context** (e.g. pipeline, service, dashboard, application, comparative analysis etc.)

4. Capstone timeline and submissions (Semester 3)

The capstone runs for **12 weeks** and has **three graded milestones**.

At the discretion of the examiner/supervisor, a viva may be called to assess the project-related work at any of the three milestones.

Milestone 1: Project Definition

Timing: Week 4

Weight: 25%

Submission (max 3 pages, fixed template):

- Problem statement
- Data source(s)
- Intended methods (linked to program modules)
- Expected outputs
- Key risks and assumptions

Purpose:

Lock scope early and confirm feasibility.

Milestone 2: Technical Checkpoint

Timing: Week 8

Weight: 25%

Submission:

- Data preprocessing completed
- Exploratory Data Analysis
- Baseline + improved method implemented
- System architecture diagram
- Code or system link
- Demo video (maximum 6 minutes)

Purpose:

Confirm technical depth and steady progress.

Milestone 3: Final Submission

Timing: Week 12

Weight: 25%

Submission:

- Final report (approx. 25–30 pages)
 - Including short reflection on limitations and trade-offs
- Code and/or system link
- Reproducibility checklist (Appendix A)

There is **no draft report review** stage.

5. Assessment summary

Component	Weight
Project Definition	25%
Technical Checkpoint	25%
Final Submission	50%
Total	100%

50% of the marks are earned **before** the final submission.

6. Supervision and support model

- A supervisor may be assigned by the main examiner. Optionally, students might be given a choice, but students should not expect this.
- Staff involvement is **evaluation-focused**, not mentoring
- Limited shared consultation sessions may be announced
- Questions should be based on published guidelines and FAQs

You should not expect:

- Regular meetings
- Iterative feedback on drafts
- Scope negotiation after approval

7. Academic integrity

Submitting someone else's work as your own is treated as gross misconduct. For each submission, there has to be sufficient evidence to prove that it's your own work. This may be further tested at a viva called by the supervisor/examiner at any stage of the project, including post-submission.

You may use tools (including AI tools) **responsibly**, but:

- You must understand and justify all decisions
- You must be able to explain your system and results
- Failure to demonstrate authorship or understanding will be penalized

The technical defense is designed to verify this.

8. Key advice for success

- Lock scope early and keep it realistic
- Focus on **depth and justification**, not breadth
- Build a working pipeline first, then improve it
- Document failures and trade-offs — they matter
- Treat this as a **professional DS/AI project**, not an assignment

Appendix A: Reproducibility Checklist (final submission)

A **Reproducibility Checklist** in the context of Data Science and AI is a standard list of items that ensures anyone reading your work can obtain the same results you did. It verifies that your project is not just a "one-off" success but a reliable scientific artefact.

In your module (CS5998), you are required to include this checklist in your **Milestone 3 (Final Report)** to confirm your work is transparent and verifiable.

What goes into the Checklist?

Typically, a reproducibility checklist asks you to confirm if you have documented the following:

1. Code & Environment

- ☐ **Dependencies listed:** Did you provide a `requirements.txt` or `environment.yml` file?
- ☐ **Hardware specified:** Did you list the GPU/CPU/RAM used? (e.g., *Google Colab T4 GPU*)
- ☐ **Random Seeds:** Did you set and report fixed random seeds (e.g., `torch.manual_seed(42)`)?

2. Data

- ☐ **Data Availability:** Is the dataset public? If not, did you explain how to access it?
- ☐ **Splits Defined:** Did you clearly state how much data was used for Training, Validation, and Testing?
- ☐ **Preprocessing:** Are the cleaning and normalization steps fully described?

3. Model & Training

- [] **Hyperparameters:** Did you list the specific learning rate, batch size, and epoch count used for your final results?
- [] **Architecture:** Is the model structure (layers, units) fully described or cited?

Appendix - B: Sample Projects

Appendix A: Example Project Types (Safe Choices)

This list is only for you to understand what an appropriate project is. You are not forced to choose a project from this list. Choosing one of these will **not guarantee high marks**, but it **guarantees acceptable scope** if executed well.

Students are also encouraged to propose their own projects, but they must meet the same depth and structure.

1. Predictive Analytics System (Tabular Data)

Typical problems

- Customer churn prediction
- Credit risk or default prediction
- Demand or sales forecasting
- Student or employee performance prediction

What the project should include

- Realistic tabular dataset
- Feature engineering and preprocessing
- Baseline model + improved model
- Model evaluation and interpretation
- Simple deployment or scoring pipeline

Common techniques

- Logistic / linear regression
- Tree-based models
- Ensemble methods
- Feature importance analysis

2. Text Analytics / NLP Pipeline

Typical problems

- Document or email classification

- Sentiment or opinion analysis
- Topic modelling and document clustering
- Resume or document information extraction

What the project should include

- Text preprocessing and representation
- Comparison of at least two approaches
- Clear evaluation strategy
- End-to-end pipeline (ingest → process → output)

Common techniques

- TF-IDF + classical ML
- Word embeddings
- Transformer-based models (where appropriate)

3. Computer Vision Application

Typical problems

- Image classification
- Object detection or segmentation
- Document or form image analysis
- Video-based detection or tracking

What the project should include

- Image/video preprocessing
- Baseline + advanced model
- Evaluation using appropriate metrics
- Discussion of data quality and limitations

Common techniques

- CNN-based models
- Transfer learning
- Classical vision + deep learning hybrids

4. Time Series Analysis and Forecasting System

Typical problems

- Sales, energy, or traffic forecasting
- Sensor or IoT data analysis
- Financial or operational time series

What the project should include

- Time-aware preprocessing

- Baseline statistical model
- ML/DL-based forecasting method
- Error analysis and comparison

Common techniques

- ARIMA / exponential smoothing
- Regression-based forecasting
- RNNs or temporal models

5. Recommendation System

Typical problems

- Product or content recommendation
- Course or resource recommendation
- User-item matching problems

What the project should include

- Data modelling (users, items, interactions)
- Baseline recommender
- Improved or hybrid recommender
- Evaluation using suitable metrics

Common techniques

- Collaborative filtering
- Content-based methods
- Matrix factorization
- Hybrid approaches

6. Anomaly / Fraud Detection System

Typical problems

- Transaction anomaly detection
- Network or system log anomalies
- Sensor fault detection

What the project should include

- Problem framing (what is “anomalous”)
- Baseline detection method
- Advanced or hybrid method
- Evaluation challenges and assumptions

Common techniques

- Statistical thresholds

- Isolation-based methods
- Autoencoders or density models

7. Business Intelligence / Decision Support System

Typical problems

- Operational dashboards
- KPI monitoring systems
- Data-driven decision support tools

What the project should include

- Data integration and cleaning
- Analytics or modelling component
- Visualization and reporting
- Clear business interpretation

Common techniques

- Aggregation and analytics
- Predictive or descriptive models
- Visualization tools and dashboards

8. End-to-End Data Engineering + Analytics Pipeline

Typical problems

- Large-scale data ingestion and processing
- Analytics pipeline on cloud or distributed systems
- Data processing for downstream ML

What the project should include

- Data ingestion and storage design
- Processing pipeline
- Analytics or ML component
- Performance or scalability discussion

Common techniques

- Distributed processing frameworks
- Cloud services
- Batch or streaming pipelines

Use of LLM API calls in the capstone project

LLM API calls may be **used**, but they must **not be the project**.

How LLM-based projects are classified

✗ Category A: Thin LLM Wrappers (NOT acceptable)

Projects that:

- Only call OpenAI / Gemini / Claude APIs
- Add prompt templates + UI
- Rely entirely on the LLM for reasoning
- Do not implement or evaluate any DS/AI technique beyond prompting

Examples

- “Chatbot for X using GPT”
- “PDF summarizer using Gemini”
- “Q&A system over documents using OpenAI”

These will **not** be approved as capstone projects.

⚠ Category B: LLM-Enhanced Applications (Conditionally acceptable)

LLMs are used as **one component**, but the project includes **substantial additional work**.

To be acceptable, the project must clearly demonstrate **at least two of the following**:

1. **Non-trivial data engineering**
 - Data cleaning, chunking strategies, indexing, metadata design
2. **Algorithmic or modeling decisions**
 - Retrieval strategies, ranking, filtering, hybrid pipelines
3. **Evaluation beyond “it looks good”**
 - Quantitative metrics, ablations, comparisons
4. **System-level design**
 - Latency handling, cost control, fallback strategies
5. **Responsible AI considerations**
 - Hallucination handling, bias analysis, privacy controls

LLM = **component**, not **solution**.

✓ Category C: LLM-Centric but Methodologically Deep (Preferred)

These projects treat LLMs as **objects of study**, not magic boxes.

Examples:

- Comparing retrieval strategies for RAG systems
- Evaluating prompt strategies vs fine-tuning (where feasible)
- Measuring hallucination rates under different constraints
- Cost–accuracy trade-off analysis across models
- Domain adaptation via data curation and retrieval

These projects are **acceptable and encouraged** *if executed rigorously*.

Mandatory requirements for any LLM-based project

If your project uses external LLM APIs, you **must** include all of the following:

1. Explicit problem framing

- What part is solved by the LLM?
- What part is solved by *your* logic?

2. Baseline without LLM (or with naive usage)

- Simple heuristic, classical method, or naive prompting
- Used for comparison

3. Evaluation

At least one of:

- Accuracy / precision / recall
- Latency
- Cost
- Failure rate / hallucination analysis

4. Cost & dependency disclosure

- Model(s) used
- Approximate API cost
- External dependency risks

5. Fallback or mitigation strategy

- What happens when the LLM fails, times out, or hallucinates?

How this is enforced

At Project Definition stage

Students must explicitly state:

- External LLM used (if any)
- Role of the LLM in the pipeline
- What **non-LLM** components they are building

Projects failing this will be **rejected early**.

During Technical Checkpoint

Students must show:

- Their own pipeline logic
- At least one comparison or ablation
- Evidence they understand failure modes

Summary: Use of external LLM APIs

Use of external LLM APIs is allowed but not sufficient.

Projects that only wrap an LLM API without substantial data processing, modeling, system design, or evaluation will not be accepted as capstone projects.

Projects that are not recommended

- Pure literature surveys
- Pure app development with minimal DS/AI content
- Overly ambitious research-level problems
- Projects requiring unavailable or sensitive data
- Thin LLM wrappers

Final note to students

A **well-executed simple project** will score higher than an **ambitious but incomplete one**.

Choose a project where you can:

- Build a complete system
- Justify decisions clearly
- Defend your work confidently in the viva